

Sprint 15 Structured Notes: Forms, Validation, and Submission Control

0. What Sprint 15 Is About

Sprint 15 is about making forms controlled, predictable, and user-friendly.

A form collects data from the user.

Example:

```
<form>
  <input>
  <button>Submit</button>
</form>
```

But a real form needs rules.

Examples of form rules:

```
Email must not be empty.
Email must look like an email.
Feedback must be at least 10 characters.
Password must be long enough.
Only company email addresses are allowed.
```

Sprint 15 teaches you how to check those rules before the form is accepted.

Main goal

Validate user input and control form submission behavior.

Main technologies in Sprint 15

Sprint 15 uses:

- HTML forms
- form input fields
- validation attributes
- JavaScript DOM selection
- events

- the `submit` event
- the `input` event
- `preventDefault()`
- the Constraint Validation API
- basic string checks
- basic regex-style validation through `pattern`
- accessible status messages

Beginner mental model

```

User types input
↓
JavaScript checks the input
↓
If invalid: show an error
↓
If valid: handle the form safely

```

Simple meaning:

A form should not blindly accept whatever the user enters. JavaScript and HTML can check the data first.

Part A: Current Progress Before Sprint 15

1. What You Already Know

Before Sprint 15, you already learned the skills needed to understand forms.

Area	What You Should Know Before Sprint 15	How It Helps in Sprint 15
Core JavaScript	Values, variables, strings, numbers, booleans, conditions, functions, arrays, loops, objects, callbacks, Sets, and Maps	Form validation uses values, conditions, strings, functions, and booleans.
Browser JavaScript	Selecting elements, changing text/content, adding/removing elements, and listening for events	Forms are HTML elements that JavaScript selects and controls.
Events	<code>click</code> , <code>input</code> , <code>change</code> , <code>keydown</code> , <code>submit</code> , and <code>preventDefault()</code>	Forms depend heavily on events.
DOM updates	<code>textContent</code> , attributes, and page messages	Validation often shows status or error messages on the page.

Area	What You Should Know Before Sprint 15	How It Helps in Sprint 15
Accessibility	Visual changes and accessibility state should stay synchronized	Form errors and success messages should be understandable to more users.

Simple summary:

Sprint 15 combines earlier browser skills into one practical task: control what happens when the user submits a form.

2. The Main Shift in Sprint 15

Before Sprint 15, you learned how to make the page interactive.

Example:

```
button.addEventListener("click", () => {
  status.textContent = "Saved!";
});
```

Sprint 15 adds a stricter question:

```
Should this form be allowed to submit?
```

The page does not only react. It checks the user's input first.

Example:

```
form.addEventListener("submit", event => {
  event.preventDefault();

  if (!email.checkValidity()) {
    email.reportValidity();
    return;
  }

  console.log("Form is valid");
});
```

Simple meaning:

When the form tries to submit, JavaScript checks whether the form data is acceptable.

Part B: Forms

3. What Is a Form?

A form is an HTML structure used to collect information from the user.

Example:

```
<form>
  <input>
  <button>Submit</button>
</form>
```

Forms are used for things like:

- login pages
- signup pages
- search boxes
- contact forms
- feedback forms
- checkout forms
- comment forms
- settings pages

Simple meaning:

A form is how a webpage asks the user for information.

4. Basic Form Example

```
<form id="loginForm">
  <label for="email">Email</label>
  <input id="email" type="email" required>

  <button type="submit">Log in</button>
</form>
```

Important parts:

Part	Meaning
<code><form></code>	The form container.
<code>id="loginForm"</code>	Lets JavaScript find the form.
<code><label></code>	Describes an input.
<code>for="email"</code>	Connects the label to the input with <code>id="email"</code> .
<code><input></code>	Lets the user enter data.
<code>type="email"</code>	Tells the browser this input should contain an email.
<code>required</code>	Tells the browser this field cannot be empty.
<code><button type="submit"></code>	Submits the form.

Beginner rule:

Use real form elements first. Do not build form behavior from random `div` elements unless you have a strong reason.

5. Why Forms Need Validation

Validation means checking whether user input follows the rules.

Example rules:

```
Email is required.
Email must look like an email.
Feedback must not be empty.
Feedback must be at least 10 characters.
A password must be at least 8 characters.
```

Without validation, bad data can enter your app.

Examples of bad data:

```
Empty email
Text that is not an email
Too-short password
Empty feedback
Unexpected format
```

Simple meaning:

Validation protects the form from accepting incomplete or incorrect input.

Part C: HTML Validation First

6. HTML Gives You Basic Validation

Before writing JavaScript validation, use HTML validation attributes.

Example:

```
<input type="email" required>
```

This means:

```
The field is required.  
The value must look like an email.
```

The browser can already check this without custom JavaScript.

Beginner rule:

Use built-in HTML validation first. Add JavaScript when you need custom behavior.

7. Common HTML Validation Attributes

```
<input required>  
<input type="email">  
<input minlength="10">  
<input maxlength="100">  
<input pattern="...">
```

Attribute	Meaning
<code>required</code>	The field cannot be empty.
<code>type="email"</code>	The value must look like an email address.
<code>minlength="10"</code>	The text must be at least 10 characters long.
<code>maxlength="100"</code>	The text cannot be longer than 100 characters.

Attribute	Meaning
<code>pattern="..."</code>	The value must match a pattern.

8. `required`

`required` means the user must enter a value.

Example:

```
<input id="username" required>
```

If the user tries to submit the form while this field is empty, the browser blocks the form.

Simple meaning:

This field cannot be blank.

9. `type="email"`

`type="email"` tells the browser that the value should look like an email address.

Example:

```
<input id="email" type="email">
```

Valid-looking examples:

```
ada@example.com  
student@school.edu  
name@company.com
```

Invalid-looking examples:

```
ada  
hello world  
student@com
```

Beginner note:

`type="email"` is useful, but it does not check every possible email rule perfectly. It gives a reasonable browser-level check.

10. `minlength`

`minlength` sets the minimum number of characters.

Example:

```
<textarea id="feedback" minlength="10"></textarea>
```

This means:

The feedback must be at least 10 characters long.

Example:

```
"Good"          -> too short  
"Very useful"   -> long enough
```

11. `maxlength`

`maxlength` sets the maximum number of characters.

Example:

```
<input id="username" maxlength="20">
```

This means:

The username cannot be longer than 20 characters.

Use `maxlength` when you want to limit how much text the user can enter.

12. pattern

`pattern` checks the input against a pattern.

Example:

```
<input id="code" pattern="[0-9]{4}">
```

This pattern means:

The value must be exactly 4 digits.

Examples:

```
1234 -> valid  
abcd -> invalid  
12   -> invalid  
12345 -> invalid
```

Beginner warning:

Do not make patterns too complex too early. Use simpler validation first when possible.

Part D: Basic Regex Awareness

13. What Is Regex?

Regex means regular expression.

Simple meaning:

A regex is a pattern used to check or search text.

In HTML forms, `pattern` uses regex-like rules.

Example:

```
<input pattern="[A-Z]{3}">
```

This means:

```
The value must contain 3 uppercase letters.
```

Examples:

```
ABC -> valid  
Abc -> invalid  
ABCD -> invalid  
12A -> invalid
```

14. Beginner Regex Rule

Use regex only when normal HTML attributes or simple string checks are not enough.

For example, do not rush into a complicated email regex.

Better beginner approach:

```
<input type="email" required>
```

Then add a simple custom JavaScript rule if needed:

```
email.value.endsWith("@company.com")
```

Simple meaning:

Let the browser check normal email format. Let JavaScript check the special company rule.

Part E: The Constraint Validation API

15. What Is the Constraint Validation API?

The Constraint Validation API is a set of browser tools that JavaScript can use to check form validity.

Simple meaning:

The browser gives JavaScript methods for asking:

Is this input valid?
Should I show the validation message?
Should I add a custom validation error?

The main methods are:

```
checkValidity()  
reportValidity()  
setCustomValidity()
```

16. `checkValidity()`

`checkValidity()` checks whether an input passes its validation rules.

Example:

```
email.checkValidity();
```

It returns:

```
true
```

if the input is valid.

It returns:

```
false
```

if the input is invalid.

Example:

```
if (email.checkValidity()) {  
  console.log("Valid email");  
} else {  
  console.log("Invalid email");  
}
```

Simple meaning:

Ask the browser: does this input pass its rules?

17. `checkValidity()` Is Silent

`checkValidity()` checks validity, but it does not necessarily show a message to the user.

Example:

```
const isValid = email.checkValidity();
console.log(isValid);
```

This is useful when your code needs a `true` or `false` answer.

Beginner rule:

Use `checkValidity()` when your JavaScript needs to make a decision.

18. `reportValidity()`

`reportValidity()` checks validity and asks the browser to show the validation message.

Example:

```
email.reportValidity();
```

Simple meaning:

Show the user what is wrong.

Example:

```
if (!email.checkValidity()) {
  email.reportValidity();
}
```

This means:

If the email is invalid, show the browser validation message.

19. `checkValidity()` vs `reportValidity()`

Method	What It Does	Use When
<code>checkValidity()</code>	Checks silently and returns <code>true</code> or <code>false</code> .	Your code needs to decide what to do.
<code>reportValidity()</code>	Checks and shows the validation message.	You want the user to see the problem.

Simple memory:

```
checkValidity() = check silently  
reportValidity() = check and report to the user
```

20. `setCustomValidity()`

`setCustomValidity()` lets you create your own validation error message.

Example:

```
email.setCustomValidity("Use a company email address.");
```

Now the input is considered invalid.

The browser will keep treating it as invalid until you clear the custom message.

21. Clearing Custom Validity

To clear a custom error, pass an empty string:

```
email.setCustomValidity("");
```

This is critical.

If you forget to clear the custom validity message, the input can stay invalid even after the user fixes the value.

Correct pattern:

```
email.setCustomValidity("");

if (!email.value.endsWith("@company.com")) {
  email.setCustomValidity("Use a company email address.");
}
```

Simple meaning:

Clear the old error first. Then check the current value.

22. Custom Validation Example

```
function validateEmail() {
  email.setCustomValidity("");

  if (email.value !== "" && !email.value.endsWith("@company.com")) {
    email.setCustomValidity("Use your company email address.");
  }

  return email.checkValidity();
}
```

Step-by-step:

1. Clear old custom errors.
 2. Check the current email value.
 3. If the value is not empty and does not end with `@company.com`, set a custom error.
 4. Return whether the email is valid.
-

Part F: The `input` Event

23. What Is the `input` Event?

The `input` event runs while the user types or changes an input value.

Example:

```
email.addEventListener("input", () => {  
  console.log(email.value);  
});
```

Simple meaning:

Every time the user changes the field, this code runs.

24. When to Use `input`

Use `input` when you want live feedback.

Examples:

- live validation
- live search
- character counters
- preview text
- checking password length while the user types

Example:

```
email.addEventListener("input", () => {  
  validateEmail();  
});
```

Simple meaning:

Validate again whenever the email changes.

25. Live Custom Validation Example

```
email.addEventListener("input", () => {  
  email.setCustomValidity("");  
  
  if (!email.value.endsWith("@company.com")) {  
    email.setCustomValidity("Use a company email address.");  
  }  
});
```

```
}  
});
```

What happens:

1. The user types.
2. The `input` event fires.
3. JavaScript clears the old error.
4. JavaScript checks the current value.
5. JavaScript sets a new error if needed.

Part G: The `submit` Event

26. What Is the `submit` Event?

The `submit` event fires when a form tries to submit.

Example:

```
form.addEventListener("submit", event => {  
  console.log("Form submitted");  
});
```

Simple meaning:

Listen for the form's submit action.

27. How a Form Can Submit

A form can submit in more than one way.

A form can submit when:

1. The user clicks a submit button.
2. The user presses Enter inside a form field.
3. JavaScript triggers form submission.

This is why you should not only listen for a button click.

Weak approach:


```
button.addEventListener("click", () => {  
  // validate  
});
```

Better approach:

```
form.addEventListener("submit", event => {  
  // validate  
});
```

Beginner rule:

Listen for `submit` on the form, not only `click` on the submit button.

Part H: `preventDefault()`

28. What Is Default Browser Behavior?

Some browser actions happen automatically.

For example, when a form submits, the browser may:

- reload the page
- navigate to another page
- send data to the URL in the form's `action`

This is normal form behavior.

29. What Does `preventDefault()` Do?

`preventDefault()` stops the browser's default action.

Example:

```
form.addEventListener("submit", event => {  
  event.preventDefault();  
});
```

Simple meaning:

Stop the browser's normal form submission for now. Let JavaScript handle it.

30. Why `preventDefault()` Matters in Sprint 15

When you are building a JavaScript-controlled form, you usually do not want the page to reload immediately.

Example:

```
form.addEventListener("submit", event => {  
  event.preventDefault();  
  console.log("Now JavaScript can validate the form.");  
});
```

If you forget `preventDefault()`, the page may reload before you see your result.

Beginner warning:

If your `console.log()` disappears quickly, or the page refreshes, check whether you forgot `event.preventDefault()`.

Part I: Full Beginner Example

31. Goal of the Example

Build a feedback form with these rules:

```
Email field:  
- required  
- must look like an email  
- must end with @company.com  
  
Feedback field:  
- required  
- minimum 10 characters  
  
Submit behavior:  
- prevent page reload  
- show validation errors
```

- show success message
- reset the form after valid submission

32. HTML Example

```
<form id="feedbackForm">
  <label for="email">Company email</label>
  <input
    id="email"
    type="email"
    required
    placeholder="name@company.com"
  >

  <label for="feedback">Feedback</label>
  <textarea
    id="feedback"
    required
    minlength="10"
    placeholder="Write at least 10 characters"
  ></textarea>

  <button type="submit">Send</button>

  <p id="status" aria-live="polite"></p>
</form>
```

33. HTML Example Explanation

```
<form id="feedbackForm">
```

Creates the form and gives it an ID so JavaScript can find it.

```
<label for="email">Company email</label>
```

Creates a label for the email input.

The `for="email"` connects this label to:

```
<input id="email">
```

```
<input id="email" type="email" required>
```

Creates an email input.

Important rules:

```
type="email"
```

checks email format.

```
required
```

blocks empty values.

```
<textarea id="feedback" required minlength="10"></textarea>
```

Creates a larger text field for feedback.

Important rules:

```
required
```

blocks empty feedback.

```
minlength="10"
```

requires at least 10 characters.

```
<button type="submit">Send</button>
```

Creates a button that submits the form.

```
<p id="status" aria-live="polite"></p>
```

Creates a status message area.

`aria-live="polite"` helps assistive technology announce updates when the message changes.

This connects Sprint 15 back to Sprint 14 accessibility work.

34. JavaScript Example

```
const form = document.getElementById("feedbackForm");
const email = document.getElementById("email");
const feedback = document.getElementById("feedback");
const status = document.getElementById("status");

function validateEmail() {
  email.setCustomValidity("");

  if (email.value !== "" && !email.value.endsWith("@company.com")) {
    email.setCustomValidity("Use your company email address.");
  }

  return email.checkValidity();
}

email.addEventListener("input", () => {
  validateEmail();
});

form.addEventListener("submit", event => {
  event.preventDefault();

  const emailIsValid = validateEmail();
  const feedbackIsValid = feedback.checkValidity();

  if (!emailIsValid) {
    email.reportValidity();
    return;
  }

  if (!feedbackIsValid) {
    feedback.reportValidity();
    return;
  }
}
```

```
status.textContent = "Form is valid. Feedback is ready to send.";
form.reset();
});
```

Part J: JavaScript Example Explanation

35. Selecting the Form

```
const form = document.getElementById("feedbackForm");
```

Finds the form in the DOM.

Simple meaning:

Store the form element in a variable named `form`.

36. Selecting the Email Input

```
const email = document.getElementById("email");
```

Finds the email input.

Simple meaning:

Store the email input element in a variable named `email`.

37. Selecting the Feedback Field

```
const feedback = document.getElementById("feedback");
```

Finds the feedback textarea.

Simple meaning:

Store the feedback field in a variable named `feedback`.

38. Selecting the Status Message

```
const status = document.getElementById("status");
```

Finds the paragraph where the success message will appear.

Simple meaning:

Store the message area in a variable named `status`.

39. Creating a Reusable Validation Function

```
function validateEmail() {
```

This creates a reusable function for email validation.

Why use a function?

Because the same email validation is needed in more than one place:

- while the user types
 - when the form submits
-

40. Clearing Old Custom Errors

```
email.setCustomValidity("");
```

This clears any old custom error.

This is important because custom validity messages stay active until cleared.

Beginner rule:

Always clear old custom validity before checking the new value.

41. Checking the Company Email Rule

```
if (email.value !== "" && !email.value.endsWith("@company.com")) {
```

This checks two things:

```
email.value !== ""
```

means:

The email field is not empty.

```
!email.value.endsWith("@company.com")
```

means:

The email does not end with @company.com.

Together, the condition means:

If the user typed something, but it is not a company email, show a custom error.

42. Setting a Custom Error

```
email.setCustomValidity("Use your company email address.");
```

This sets a custom validation message.

Now the input is invalid until the custom message is cleared.

43. Returning the Validity Result

```
return email.checkValidity();
```


This returns either:

```
true
```

or:

```
false
```

Simple meaning:

The function gives back the answer: is the email valid right now?

44. Validating While the User Types

```
email.addEventListener("input", () => {  
  validateEmail();  
});
```

This runs `validateEmail()` every time the email input changes.

Simple meaning:

Keep the validation state updated as the user types.

45. Listening for Form Submission

```
form.addEventListener("submit", event => {
```

This listens for the form's `submit` event.

This is better than listening only for a button click because forms can also submit when the user presses Enter.

46. Stopping the Default Submit Behavior

```
event.preventDefault();
```

This stops the browser from reloading or navigating away.

Simple meaning:

Pause the normal form submission so JavaScript can validate first.

47. Checking Both Fields

```
const emailIsValid = validateEmail();  
const feedbackIsValid = feedback.checkValidity();
```

This checks:

- whether the email is valid
- whether the feedback field is valid

`validateEmail()` checks the email's built-in rules and custom company rule.

`feedback.checkValidity()` checks the feedback field's HTML rules:

```
required  
minlength="10"
```

48. Handling an Invalid Email

```
if (!emailIsValid) {  
  email.reportValidity();  
  return;  
}
```

This means:

If the email is invalid, show the validation message and stop the function.

The `return` is important.

It prevents the success code from running when the email is invalid.

49. Handling Invalid Feedback

```
if (!feedbackIsValid) {  
    feedback.reportValidity();  
    return;  
}
```

This means:

If the feedback is invalid, show the validation message and stop the function.

Again, `return` prevents the success code from running too early.

50. Showing a Success Message

```
status.textContent = "Form is valid. Feedback is ready to send.";
```

This changes the status paragraph text.

Use `textContent` because this is normal text.

Beginner safety rule:

Use `textContent` for normal messages and user-controlled text. Avoid `innerHTML` unless you intentionally need HTML and fully control the content.

51. Resetting the Form

```
form.reset();
```

This clears the form fields after successful validation.

Simple meaning:

After the form is valid and handled, empty the form again.

Part K: Form Submission Settings

52. `action`

The `action` attribute tells the browser where to send form data.

Example:

```
<form action="/feedback">
```

Simple meaning:

Send the form data to `/feedback` when the form submits.

If there is no `action`, the form usually submits to the current page URL.

53. `method`

The `method` attribute controls how the form data is sent.

Common values:

```
<form method="GET">
```

and:

```
<form method="POST">
```

54. `GET`

`GET` usually puts form data in the URL.

Example idea:

```
/search?q=javascript
```

Use `GET` for actions like:

- searching
- filtering
- reading data
- requesting information

Simple meaning:

`GET` is usually for asking for data.

55. `POST`

`POST` sends form data in the request body.

Use `POST` for actions like:

- creating an account
- sending feedback
- submitting a checkout form
- creating or updating data

Simple meaning:

`POST` is usually for sending data that creates or changes something.

56. `GET` vs `POST`

Method	Simple Meaning	Common Use
<code>GET</code>	Ask for data	Search, filters, reading data
<code>POST</code>	Send data to create or update something	Signup, feedback, checkout, saving changes

Beginner memory:

GET = ask for data
POST = send data

57. `enctype`

`enctype` controls how form data is encoded before being sent.

Common values:

Value	Use
<code>application/x-www-form-urlencoded</code>	Default form encoding.
<code>multipart/form-data</code>	Used for file uploads.
<code>text/plain</code>	Plain text form data; uncommon.

Beginner rule:

Use this when uploading files:

```
enctype="multipart/form-data"
```

For normal beginner forms, you usually do not need to set `enctype` manually.

Part L: Accessibility in Sprint 15

58. Why Form Accessibility Matters

Validation is not only about blocking bad input.

Good validation should also help the user understand what happened.

For example:

```
<p id="status" aria-live="polite"></p>
```

Then JavaScript can update it:

```
status.textContent = "Form is valid. Feedback is ready to send.";
```

Because the paragraph has `aria-live="polite"`, assistive technology may announce the change.

Simple meaning:

If JavaScript changes an important message, make sure users can notice it.

59. `aria-live="polite"`

`aria-live="polite"` tells assistive technology:

This content may change. Announce the update when it is reasonable to do so.

Common uses:

- success messages
- form validation messages
- saved messages
- loading messages
- status updates

Example:

```
<p id="status" aria-live="polite"></p>
```

Beginner rule:

Use a live region for important dynamic messages that appear after user action.

Part M: Common Sprint 15 Mistakes

60. Mistake 1: Only Listening for Button Clicks

Weak version:

```
button.addEventListener("click", () => {  
  // validate  
});
```

Problem:

The form can submit without a button click, such as when the user presses Enter.

Better version:

```
form.addEventListener("submit", event => {  
  // validate  
});
```

Beginner rule:

Validate on the form's `submit` event.

61. Mistake 2: Forgetting `preventDefault()`

Weak version:

```
form.addEventListener("submit", event => {  
  console.log("Submitted");  
});
```

Problem:

The page may reload.

Better version:

```
form.addEventListener("submit", event => {  
  event.preventDefault();  
  console.log("Submitted");  
});
```

Beginner rule:

If JavaScript is controlling the form, call `event.preventDefault()` first.

62. Mistake 3: Not Clearing Custom Validity

Weak version:

```
email.setCustomValidity("Invalid email");
```

Problem:

If you never clear the error, the input can stay invalid.

Better version:

```
email.setCustomValidity("");  
  
if (!email.value.endsWith("@company.com")) {  
    email.setCustomValidity("Use a company email address.");  
}
```

Beginner rule:

Clear old custom errors before setting new ones.

63. Mistake 4: Making Regex Too Complex Too Early

Weak approach:

```
<input pattern="very-complicated-email-regex">
```

Problem:

Complicated regex can be hard to read, hard to debug, and still not perfect.

Better beginner approach:

```
<input type="email" required>
```

Then add a simple custom rule if needed:

```
email.value.endsWith("@company.com")
```

Beginner rule:

Use browser validation first. Use simple custom checks second. Use complex regex only when necessary.

64. Mistake 5: Showing User Input with `innerHTML`

Weak version:

```
status.innerHTML = feedback.value;
```

Problem:

`innerHTML` treats the value as HTML.

Better version:

```
status.textContent = feedback.value;
```

Beginner rule:

Use `textContent` for user-controlled text.

65. Mistake 6: Forgetting `return` After Invalid Input

Weak version:

```
if (!emailIsValid) {  
    email.reportValidity();  
}  
  
status.textContent = "Success";
```

Problem:

The success message may still run even when the email is invalid.

Better version:

```
if (!emailIsValid) {  
    email.reportValidity();  
    return;  
}  
  
status.textContent = "Success";
```

Beginner rule:

After showing an error, use `return` to stop the submit handler.

66. Mistake 7: Validating Only Once

Weak version:

```
const emailIsValid = email.checkValidity();
```

Then never checking again.

Problem:

The user can change the input after the first check.

Better approach:

```
email.addEventListener("input", () => {
  validateEmail();
});

form.addEventListener("submit", event => {
  event.preventDefault();
  validateEmail();
});
```

Beginner rule:

Validate when the input changes and again when the form submits.

Part N: Core Sprint 15 Patterns

67. Basic Submit Pattern

```
form.addEventListener("submit", event => {
  event.preventDefault();

  if (!input.checkValidity()) {
    input.reportValidity();
    return;
  }
});
```

```
    console.log("Form is valid");  
  });
```

This is the minimum pattern you should understand.

Step-by-step:

1. Listen for form submission.
2. Stop the browser's default behavior.
3. Check whether the input is valid.
4. If invalid, show the problem and stop.
5. If valid, continue.

68. Custom Validation Pattern

```
function validateInput() {  
  input.setCustomValidity("");  
  
  if (input.value !== "expected value") {  
    input.setCustomValidity("Custom error message.");  
  }  
  
  return input.checkValidity();  
}
```

Simple meaning:

Use a function to keep custom validation organized.

69. Live Validation Pattern

```
input.addEventListener("input", () => {  
  validateInput();  
});
```

Simple meaning:

Check the input again while the user types.

70. Submit Validation Pattern with Multiple Fields

```
form.addEventListener("submit", event => {
  event.preventDefault();

  const firstIsValid = firstInput.checkValidity();
  const secondIsValid = secondInput.checkValidity();

  if (!firstIsValid) {
    firstInput.reportValidity();
    return;
  }

  if (!secondIsValid) {
    secondInput.reportValidity();
    return;
  }

  status.textContent = "Form is valid.";
});
```

Simple meaning:

Check each important field. Stop at the first invalid field.

Part O: Key Terms

71. Form

A form is an HTML structure that collects user input.

Example:

```
<form></form>
```

72. Input

An input is a form field where the user enters data.

Example:

```
<input id="email">
```

73. Validation

Validation means checking whether user input follows the rules.

Example:

```
Email must not be empty.
```

74. Constraint Validation API

The browser's built-in JavaScript tools for checking form validity.

Examples:

```
checkValidity()  
reportValidity()  
setCustomValidity()
```

75. `checkValidity()`

Checks whether an input is valid and returns `true` or `false`.

76. `reportValidity()`

Shows the validation message to the user.

77. `setCustomValidity()`

Sets a custom validation error message.

78. `setCustomValidity("")`

Clears a custom validation error message.

79. `input` Event

Runs when the user changes an input value.

80. `submit` Event

Runs when the form tries to submit.

81. `preventDefault()`

Stops the browser's default behavior, such as reloading the page after form submission.

82. `form.reset()`

Clears the form fields.

83. `action`

Tells the form where to send data.

84. `method`

Tells the form how to send data.

Common values:

`GET`
`POST`

85. `enctype`

Tells the form how to encode data before sending it.

Important for file uploads:

```
enctype="multipart/form-data"
```

Part P: Minimum Passing Standard for Sprint 15

86. You Are Ready to Move On When You Can Explain This

```
form.addEventListener("submit", event => {  
  event.preventDefault();  
  
  if (!input.checkValidity()) {  
    input.reportValidity();  
    return;  
  }  
  
  console.log("Form is valid");  
});
```

You should be able to explain each part.

87. What `submit` Means

```
form.addEventListener("submit", ...)
```

means:

```
Run this code when the form tries to submit.
```

88. What `preventDefault()` Means

```
event.preventDefault();
```

means:

Stop the browser's normal form submission behavior.

89. What `checkValidity()` Means

```
input.checkValidity()
```

means:

Return true if the input is valid, false if it is invalid.

90. What `reportValidity()` Means

```
input.reportValidity()
```

means:

Show the validation problem to the user.

91. What `setCustomValidity()` Means

```
input.setCustomValidity("Error message")
```

means:

Set a custom validation error.

92. What `setCustomValidity("")` Means

```
input.setCustomValidity("")
```

means:

```
Clear the custom validation error.
```

Part Q: Practice Task

93. Build a Feedback Form

Build a feedback form with the following behavior.

Email field

Rules:

- required
- type email
- must end with @company.com

Suggested HTML:

```
<input id="email" type="email" required>
```

Suggested custom JavaScript rule:

```
email.value.endsWith("@company.com")
```

Feedback field

Rules:

- required
- minimum 10 characters

Suggested HTML:

```
<textarea id="feedback" required minlength="10"></textarea>
```

Submit behavior

Rules:

- prevent page reload
- show validation errors
- show success message in an aria-live region

Suggested status HTML:

```
<p id="status" aria-live="polite"></p>
```

94. Complete Practice Solution

```
<form id="feedbackForm">
  <label for="email">Company email</label>
  <input id="email" type="email" required>

  <label for="feedback">Feedback</label>
  <textarea id="feedback" required minlength="10"></textarea>

  <button type="submit">Send</button>

  <p id="status" aria-live="polite"></p>
</form>

<script>
  const form = document.getElementById("feedbackForm");
  const email = document.getElementById("email");
  const feedback = document.getElementById("feedback");
  const status = document.getElementById("status");
```

```

function validateEmail() {
  email.setCustomValidity("");

  if (email.value !== "" && !email.value.endsWith("@company.com")) {
    email.setCustomValidity("Use your company email address.");
  }

  return email.checkValidity();
}

email.addEventListener("input", () => {
  validateEmail();
});

form.addEventListener("submit", event => {
  event.preventDefault();

  const emailIsValid = validateEmail();
  const feedbackIsValid = feedback.checkValidity();

  if (!emailIsValid) {
    email.reportValidity();
    return;
  }

  if (!feedbackIsValid) {
    feedback.reportValidity();
    return;
  }

  status.textContent = "Form is valid. Feedback is ready to send.";
  form.reset();
});
</script>

```

Part R: Sprint 15 Summary

95. Main Ideas to Remember

Sprint 15 teaches you that forms should be checked before they are accepted.

The main flow is:

```
User submits form
↓
JavaScript stops default browser behavior
↓
JavaScript checks input validity
↓
Invalid input shows an error
↓
Valid input continues safely
```

96. Most Important Tools

Tool	Purpose
<code>required</code>	Makes a field mandatory.
<code>type="email"</code>	Checks email-like format.
<code>minlength</code>	Requires a minimum length.
<code>maxlength</code>	Limits maximum length.
<code>pattern</code>	Checks a text pattern.
<code>checkValidity()</code>	Returns whether the input is valid.
<code>reportValidity()</code>	Shows validation errors.
<code>setCustomValidity()</code>	Sets a custom validation error.
<code>setCustomValidity("")</code>	Clears a custom validation error.
<code>input</code> event	Runs while the user changes a field.
<code>submit</code> event	Runs when the form tries to submit.
<code>preventDefault()</code>	Stops the browser's normal submit behavior.
<code>form.reset()</code>	Clears the form fields.
<code>textContent</code>	Safely writes text to the page.
<code>aria-live="polite"</code>	Helps announce dynamic status messages.

97. Final Beginner Rule

Use this order when building beginner forms:

1. Use correct HTML form elements.
2. Add built-in HTML validation attributes.
3. Add JavaScript only for custom validation or custom submit behavior.
4. Listen for `submit` on the form.
5. Use `preventDefault()` when JavaScript controls submission.
6. Use `checkValidity()` and `reportValidity()` to work with browser validation.
7. Use `setCustomValidity()` for custom errors.
8. Clear custom errors with `setCustomValidity("")`.
9. Show messages with `textContent`.
10. Use `aria-live` for important dynamic status messages.

Simple final memory:

HTML defines the rules.
JavaScript controls when and how the rules are checked.
The user should clearly understand what went wrong or what succeeded.